Mi Día: de la idea a tu primera aplicación web

Guía de estudio y práctica

Recorrido del trabajo realizado el 2 y 3 de noviembre de 2026.

Esta guía explica cómo pasamos de un repositorio con un README a una aplicación de tareas que puedes abrir en tu laptop ASUS. Incluye la preparación del entorno, la creación, las pruebas, la investigación del fallo, la subida a GitHub y el uso en Windows.

Está pensada para una persona que empieza desde cero. No necesitas memorizar comandos: primero comprende para qué sirve cada paso y después repítelo con calma.

El resultado: Mi Día, una aplicación en español que permite agregar, completar, desmarcar y eliminar tareas. Incluye filtros, progreso y guardado en el navegador. No requiere cuentas ni instalación de paquetes para usarla.

Repositorio: https://github.com/wcorporan/mi-primera-app Rama de la aplicación: mi-dia-corregida-20261103 Versión visible: 1.1 · Lista para usar

Qué encontrarás:

1. Lo que querías construir y el entorno inicial.
2. Cómo se creó la aplicación.
3. Cómo guarda y muestra las tareas.
4. El problema de los botones y su investigación.
5. Las pruebas de funcionamiento.
6. GitHub y la subida de la versión corregida.
7. Descargar y abrir la aplicación en Windows.
8. Uso diario, actualizaciones y solución de problemas.
9. Cómo repetir el proceso y vocabulario básico.

Esta guía resume lo que hicimos; no es una transcripción literal del chat. Distingue lo comprobado de las causas que solo fueron posibles explicaciones.

1. El punto de partida

Primero pediste configurar un entorno en la nube para el repositorio wcorporan/mi-primera-app, en la rama main. Usamos la habilidad cloud-environment-onboarding:setup para inspeccionar y preparar el entorno.

Un entorno en la nube es una computadora de trabajo remota. Permite revisar archivos, ejecutar herramientas y comprobar una aplicación sin instalar todo en tu ASUS. En este caso, el proyecto estaba en /workspace/mi-primera-app. Esa ruta pertenece a la computadora remota: no es una carpeta de tu laptop.

Revisamos los archivos y el estado de Git. El repositorio solo contenía README.md, con el nombre del proyecto. No había aplicación, dependencias ni pruebas. Por eso no instalamos paquetes ni inventamos instrucciones de arranque que todavía no hacían falta.

Comprobamos que estaban disponibles Git, Node.js, npm y Python. Tener estas herramientas disponibles no significa que todas sean necesarias para usar la aplicación final.

También verificamos el acceso de lectura a GitHub. La referencia main coincidía con el commit inicial del repositorio. Una primera comprobación encontró una restricción local de ejecución; al usar la vía autorizada del entorno, el acceso funcionó. No fue necesario pedirte un token de GitHub.

Resultado de esta etapa:

- El repositorio estaba accesible y sin cambios de aplicación.
- El entorno tenía herramientas suficientes para desarrollar y comprobar una web sencilla.
- Aún no existía ninguna aplicación que abrir o probar.
- No se guardaron scripts de instalación o arranque, porque el contenido inicial no los necesitaba.

Lección: configurar el entorno, crear una aplicación y subirla a GitHub son acciones distintas. Completar una no implica haber completado las otras.

2. De tu idea a los archivos

Después pediste una aplicación de tareas diarias, completamente en español, con diseño moderno y adaptable a computadora y celular. Debía permitir agregar, completar y eliminar tareas, y conservarlas después de cerrar la página.

Elegimos HTML, CSS y JavaScript, sin un framework ni dependencias de ejecución. Para este tamaño de proyecto, permiten conseguir lo necesario con pocos archivos y sin instalar herramientas en tu laptop.

Cada tecnología tiene una función:

- HTML define la estructura: títulos, campo de texto, formulario, botones y lista.
- CSS define el aspecto: colores, espacios, tamaños y adaptación a pantallas pequeñas.
- JavaScript da comportamiento: recibe las acciones, cambia las tareas y las guarda.

La primera versión tenía estos archivos:

- index.html: página principal y textos en español.
- estilos.css: diseño y adaptación a computadora y celular.
- app.js: lógica de tareas, filtros, progreso y almacenamiento.
- README.md: instrucciones para abrirla y guardar los cambios.

Añadimos un campo «¿Qué quieres hacer hoy?», el botón «Agregar tarea», una lista de tareas y una papelera para cada una. También incorporamos filtros «Todas», «Pendientes» y «Completadas», un contador y una barra de progreso.

La fecha se muestra en español con la fecha local del navegador. El diseño usa una tarjeta central, tonos verdes y espacios amplios. En pantallas estrechas, el campo y el botón se colocan uno debajo del otro.

La aplicación no envía tareas a un servidor. Tampoco necesita una base de datos externa ni una cuenta de usuario. Esa sencillez facilita abrirla como archivo local.

En la versión final, el diseño y JavaScript se integraron en index.html. El motivo de este cambio se explica en la sección 4. La aplicación actual se puede utilizar con ese único archivo.

3. Cómo funciona por dentro

Cuando escribes una tarea y pulsas Enter o «Agregar tarea», el formulario ejecuta una función de JavaScript. Ambas acciones usan el mismo evento del formulario, llamado submit; así comparten la misma lógica.

El código evita que el formulario recargue la página. Luego quita los espacios al principio y al final del texto y comprueba que la tarea no esté vacía. El campo admite hasta 200 caracteres al escribir normalmente en la interfaz.

Cada tarea contiene tres datos:

- id: identificador para distinguirla de las demás.
- texto: lo que escribiste.
- completada: verdadero o falso, según su estado.

Ejemplo conceptual:

```
{
  "id": "identificador-de-la-tarea",
  "texto": "Salir a caminar",
  "completada": false
}
```

Al marcar una casilla, cambia el estado de esa tarea. Al pulsar la papelera, se elimina de la lista. Después de cada cambio, la aplicación actualiza lo que ves y guarda la lista.

El guardado usa localStorage, un espacio que ofrece el navegador para conservar datos entre sesiones. La lista se convierte a texto con JSON y se almacena con la clave mi-dia.tareas.v1. Al abrir de nuevo, el código lee ese texto y reconstruye las tareas.

Es importante entender sus límites:

- Tus tareas pertenecen a ese navegador y perfil; no se sincronizan entre equipos.
- Edge y Chrome pueden tener listas diferentes.
- Borrar los datos del navegador puede borrar la lista.
- El modo privado no es adecuado para conservar tareas de forma duradera.
- Mover o renombrar un archivo local puede cambiar el acceso a los datos, según el navegador.

Si el navegador bloquea el almacenamiento, se muestra un aviso. La lista sigue funcionando durante esa sesión, pero no se promete conservarla al cerrar.

Las tareas se insertan como texto, no como código HTML. Por ejemplo, escribir una etiqueta HTML no debe crear elementos ni ejecutar instrucciones en la página.

4. El problema: se veía bien, pero no agregaba tareas

Informaste que el diseño aparecía, pero Enter y «Agregar tarea» no creaban tareas. Una página puede mostrar su HTML y CSS aunque su JavaScript no se haya ejecutado. Por eso ver el diseño no confirma que los controles estén funcionando.

Revisamos la carga del archivo app.js, los identificadores de los controles y la conexión del evento submit. En el entorno de pruebas, la versión original funcionaba con los archivos completos. No encontramos una prueba que permitiera afirmar cuál era la causa exacta en tu ASUS.

Sí reprodujimos el mismo síntoma al bloquear intencionadamente la petición de app.js: el diseño aparecía y el formulario no creaba tareas. Esto demostró que la falta de carga del código podía explicar el comportamiento, pero no confirmó por sí sola lo sucedido en tu laptop.

Para eliminar esa dependencia, integramos el diseño y el JavaScript dentro de index.html. El código está al final de la página, después de los controles, para que estos ya existan al conectarlos con sus funciones.

Además, incorporamos estas señales:

- El campo y el botón permanecen desactivados hasta terminar la inicialización.
- Si el código no se ejecuta, queda visible un aviso de que la aplicación no está lista.
- El pie muestra «Versión 1.1 · Lista para usar» cuando la inicialización termina.
- Al agregar, completar o eliminar, aparece una confirmación de la acción.

También encontramos que el primer ZIP preparado aún contenía la versión antigua. Actualizamos las dos copias del ZIP, pero surgió otro problema: los enlaces a archivos del entorno remoto no iniciaban una descarga accesible desde tu interfaz de Codex.

La solución de entrega fue subir la aplicación a GitHub y descargarla desde allí. Lección: hay que comprobar tanto que la aplicación funciona como que la persona recibe realmente la versión correcta.

5. Cómo comprobamos que funcionaba

Usamos Chromium, un navegador de la misma familia tecnológica que Chrome y Edge, controlado por Playwright. Las pruebas se ejecutaron con un servidor local temporal y un perfil de navegador separado. No usaron tus tareas personales.

Comprobamos estas acciones:

- Agregar una tarea haciendo clic en «Agregar tarea».
- Agregar otra pulsando Enter.
- Rechazar una entrada que solo contenía espacios.
- Marcar una tarea como completada y volver a dejarla pendiente.
- Mostrar correctamente los filtros y el progreso.
- Eliminar una tarea y comprobar que no reaparecía al recargar.
- Cerrar completamente el navegador de prueba y volver a abrirlo con el mismo perfil para recuperar la lista y su estado.

También comprobamos situaciones especiales:

- Texto que parece HTML se muestra como texto sin crear elementos ejecutables.
- Datos guardados dañados producen un aviso en lugar de impedir usar la página.
- Si se bloquea el almacenamiento, se informa de que los cambios serán temporales.
- Con JavaScript deshabilitado, aparece el aviso de inicio y el formulario queda desactivado.
- La versión integrada no necesita pedir archivos JavaScript externos.
- No hay desbordamiento horizontal en anchos de 1280, 390 y 320 píxeles.

Se revisó la sintaxis del JavaScript y la ausencia de errores de espacios en los cambios de Git. La prueba reutilizable se guardó en pruebas/comprobar_app.py. Para ejecutarla hacen falta herramientas de desarrollo; no las necesitas para usar Mi Día.

Límite de la comprobación: las pruebas automatizadas se ejecutaron en el entorno remoto, mediante un servidor local. No fueron una prueba automática de tu Windows ni de la apertura directa de archivos en tu ASUS.

Tu comprobación práctica completó esa parte: descargaste desde GitHub, abriste la aplicación, creaste una tarea y confirmaste que seguía guardada después de cerrar y volver a abrir.

Para repetir la prueba tú mismo: agrega dos tareas, completa una, elimina la otra, cierra la página y vuelve a abrir el mismo archivo con el mismo navegador. Debe quedar la tarea completada.

6. Cómo guardamos el trabajo en GitHub

Al principio, los archivos solo estaban en la computadora remota de Codex. Que una aplicación exista allí no significa que ya esté en GitHub ni que puedas descargarla desde tu laptop.

Después autorizaste expresamente subir la versión corregida a una rama nueva. Una rama es una línea de trabajo separada. Nos permitió guardar la aplicación sin cambiar main, la rama original del repositorio.

Creamos la rama mi-dia-corregida-20261103. Incluimos estos archivos en la subida inicial:

- index.html: aplicación completa.
- README.md: instrucciones y explicaciones.
- pruebas/comprobar_app.py: comprobación automatizada.

Git registró el commit 98e65f5, con el mensaje «Crear Mi Día con tareas persistentes y JavaScript integrado». Un commit es un punto guardado del historial; push envía ese historial a GitHub.

Los pasos principales fueron:

```
git switch -c mi-dia-corregida-20261103
git add index.html README.md pruebas/comprobar_app.py
git commit -m "Crear Mi Día con tareas persistentes y JavaScript integrado"
git push -u origin mi-dia-corregida-20261103
```

Después verificamos que GitHub tenía el mismo commit de la nueva rama y que main seguía en el commit inicial 13cccd2. No combinamos las ramas ni publicamos un sitio en GitHub Pages.

Para ver la aplicación guardada, abre:
https://github.com/wcorporan/mi-primera-app/tree/mi-dia-corregida-20261103

Selecciona siempre la rama correcta antes de descargar. Si eliges main, puedes obtener el contenido inicial en lugar de la aplicación. Ver archivos en GitHub tampoco significa ejecutar la aplicación: en este flujo la descargamos y la abrimos en el navegador.

Este documento se añade después a la misma rama. Su historial tendrá un commit adicional de documentación; eso no cambia cuál fue el commit inicial de la aplicación.

7. Descargar y abrir en tu ASUS

Estos son los pasos completos que conviene repetir cuando quieras descargar el proyecto desde GitHub.

1. Abre el enlace de la rama corregida de la sección anterior. Inicia sesión en GitHub si el repositorio lo requiere.
2. Comprueba que el selector de rama diga mi-dia-corregida-20261103.
3. Pulsa el botón verde Code y después Download ZIP.
4. Abre el Explorador de archivos con Windows + E y entra en Descargas.
5. Busca un ZIP cuyo nombre contenga mi-primera-app. Windows puede ocultar la extensión .zip.
6. Haz clic derecho sobre él y selecciona Extraer todo… Después pulsa Extraer.
7. Abre la carpeta extraída. Si dentro hay otra carpeta, entra hasta encontrar index.html.
8. Haz doble clic en index.html y elige Edge o Chrome si Windows te pregunta con qué abrirlo.
9. Busca «Versión 1.1 · Lista para usar» al pie de la aplicación.
10. Agrega una tarea para comprobarla.

No abras el archivo desde la vista interior del ZIP. Primero extrae el contenido a una carpeta normal.

Si necesitas reemplazar una copia anterior:

1. Cierra la pestaña anterior de Mi Día.
2. Selecciona el index.html nuevo y pulsa Ctrl + C.
3. Abre la carpeta donde estaba la aplicación que usabas.
4. Pulsa Ctrl + V y elige Reemplazar el archivo en el destino.
5. Vuelve a abrir ese index.html con el navegador y perfil de antes.

Reemplazar el archivo en la misma ubicación reduce el riesgo de terminar con varias copias y listas distintas. No borres los datos del navegador.

Si el ZIP no aparece, pulsa Ctrl + J en Edge o Chrome para revisar el historial de descargas. «Mostrar en carpeta» lleva a la ubicación real del archivo. Un enlace del chat a una ruta /workspace no garantiza una descarga en la laptop: por eso terminamos usando GitHub.

8. Usar Mi Día todos los días

Para regresar a la aplicación, abre el mismo index.html que probaste. No necesitas descargar el ZIP cada día ni volver a GitHub. Las tareas se mantienen en el navegador, no dentro del ZIP.

Puedes crear un acceso directo en el escritorio:

1. Busca index.html en la carpeta extraída.
2. Haz clic derecho sobre él.
3. En Windows 11, selecciona Mostrar más opciones si hace falta.
4. Elige Enviar a → Escritorio (crear acceso directo).
5. Cambia el nombre del acceso a Mi Día si quieres.

El acceso directo es un enlace al archivo original. No borres ni muevas su carpeta, porque dejaría de encontrarlo. Si Windows pregunta qué navegador usar, elige el mismo donde guardaste tus tareas.

Uso diario:

- Escribe una tarea y pulsa Enter o Agregar tarea.
- Marca su casilla al terminar; quedará tachada.
- Desmarca la casilla si quieres volver a dejarla pendiente.
- Usa los filtros para revisar tu lista.
- Pulsa la papelera para eliminar. La versión actual no tiene una función de deshacer.

Si no agrega tareas, comprueba primero la etiqueta de versión y el aviso de inicio. Abre el archivo extraído en una pestaña normal; una vista previa puede restringir JavaScript. Si ocurre un fallo, toma una captura e indica el navegador, cómo abriste el archivo y qué sucedió al pulsar el botón.

Si desaparecen tareas, revisa si cambiaste de navegador, de perfil, de carpeta o al modo privado. No borres datos para intentar recuperarlas. Esta aplicación no tiene copia de seguridad ni sincronización automática; GitHub guarda el código, no tu lista personal.

No es necesario instalar un servidor para el uso que probaste. Como opción de desarrollo, si ya tienes Python puedes ejecutar en la carpeta del proyecto:

```
python -m http.server 8000 --bind 127.0.0.1
```

En Windows también puede usarse py en lugar de python. La dirección sería http://127.0.0.1:8000. Este modo tiene su propio almacenamiento; no importa automáticamente las tareas del archivo local. Detén el servidor con Ctrl + C.

9. Cómo repetir este proceso en otro proyecto

1. Define una meta pequeña. Por ejemplo: «Una web en español para guardar una lista de compras». Describe acciones concretas, no solo su apariencia.
2. Crea o selecciona un repositorio y comprueba qué contiene. Lee las instrucciones existentes antes de modificar archivos.
3. Prepara el entorno. Instala solo las herramientas y dependencias que el proyecto necesite. Reutiliza versiones e instrucciones existentes.
4. Elige la tecnología más sencilla que cumpla el objetivo. Para una web local pequeña, HTML, CSS y JavaScript pueden ser suficientes.
5. Construye primero el flujo principal. Comprueba que agregar, editar o eliminar funciona antes de ampliar funciones.
6. Decide cómo guardar los datos y explica sus límites. Guardado local no significa sincronización ni copia de seguridad.
7. Prueba casos reales: entrada válida, entrada vacía, recarga, cierre y reapertura. Verifica el comportamiento, no solo la apariencia.
8. Si falla, reproduce el error y revisa la consola y las peticiones de archivos. No presentes una hipótesis como causa confirmada.
9. Guarda una versión en Git y súbela a una rama nueva de GitHub. Verifica que la subida terminó y que contiene los archivos esperados.
10. Entrega instrucciones y un acceso comprobable. Descarga desde la rama correcta, extrae el ZIP y abre el archivo. Identifica visualmente la versión.

Solicitud que puedes reutilizar: «Crea en este repositorio una aplicación web sencilla en español para [objetivo]. Debe permitir [acciones], funcionar en computadora y celular y guardar los datos en [lugar]. Revisa las instrucciones existentes, crea los archivos necesarios, ejecuta pruebas reales y explica cómo abrirla en Windows. Antes de subirla a GitHub, indícame la rama que vas a usar».

Solicitud para investigar un fallo: «Uso [navegador] y abro la aplicación desde [archivo o dirección]. Al hacer [acción], espero [resultado], pero ocurre [problema]. Esta es la captura o el mensaje. Investiga, corrige y vuelve a probar el flujo completo».

Pequeño ejercicio: pide cambiar el título o el color en una rama nueva. Verifica el resultado y conserva el guardado antes de añadir funciones mayores.

10. Vocabulario y repaso final

Aplicación web: programa que usas dentro de un navegador. HTML: estructura y contenido de la página. CSS: reglas de aspecto y adaptación visual. JavaScript: código que responde a tus acciones. Navegador: programa como Edge o Chrome que abre la web. localStorage: espacio de guardado local ofrecido por el navegador. JSON: formato de texto usado para representar datos. Repositorio: carpeta de proyecto con historial de cambios. Git: herramienta que registra y organiza esos cambios. GitHub: servicio donde alojamos el repositorio en Internet. Rama: línea de trabajo separada dentro del repositorio. main: nombre de la rama original de este proyecto. Commit: punto guardado en el historial. Push: envío de commits a GitHub. Pull request: propuesta para revisar y combinar una rama con otra. ZIP: archivo comprimido que contiene otros archivos. Extraer: sacar los archivos del ZIP a una carpeta normal. Acceso directo: enlace para abrir rápidamente un archivo ubicado en otra carpeta. README: documento de instrucciones del proyecto. Prueba automatizada: programa que ejecuta acciones y comprueba resultados.

Lo que conseguiste:

- Definir tu primera aplicación web y preparar un entorno de desarrollo.
- Crear una interfaz en español y adaptable a distintos tamaños.
- Guardar tareas en el navegador y entender los límites de ese guardado.
- Investigar un fallo separando apariencia, carga del código y comportamiento.
- Guardar el proyecto en una rama nueva de GitHub sin modificar main.
- Descargar, extraer y abrir la aplicación en tu ASUS.
- Confirmar que una tarea seguía guardada después de cerrar y abrir.

Preguntas para estudiar:

1. ¿Por qué ver el diseño no garantiza que los botones funcionen?
2. ¿Qué diferencia hay entre guardar el código en GitHub y guardar tareas en el navegador?
3. ¿Para qué usamos una rama nueva?
4. ¿Por qué extraemos el ZIP antes de abrir index.html?
5. ¿Por qué conviene abrir siempre el mismo archivo con el mismo navegador?

Respuestas breves: el JavaScript puede no ejecutarse; son lugares y datos distintos; para conservar main; para trabajar con archivos normales; para volver al mismo contexto de almacenamiento.

Enlace de referencia:
https://github.com/wcorporan/mi-primera-app/tree/mi-dia-corregida-20261103